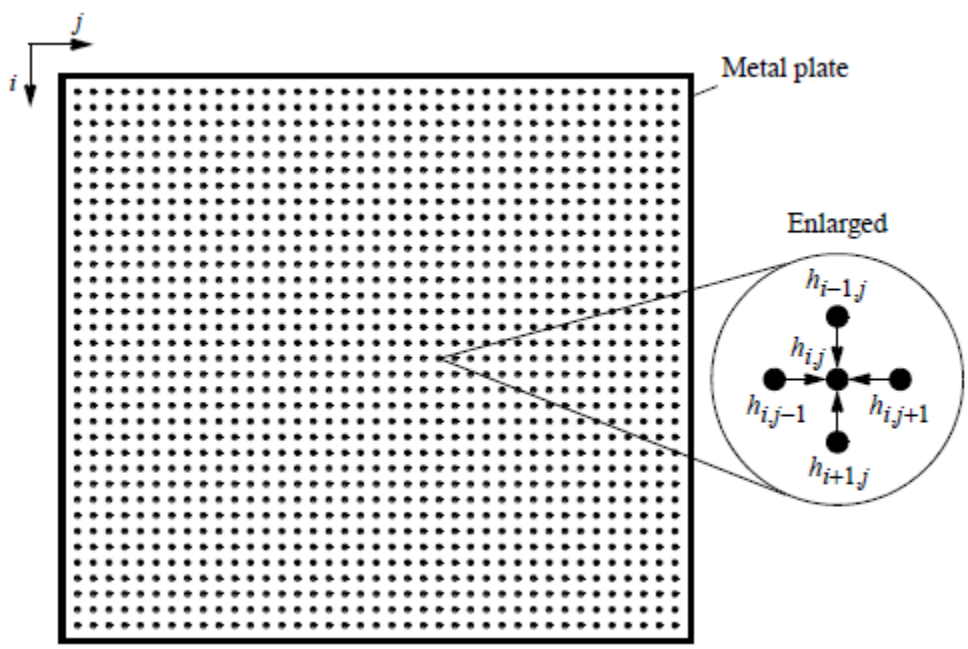
Multicore Processors
Lab Assignment # 2

In this programming assignment, you will implement a parallel application with OpenMP and see yourself when it yields better performance than sequential version and when not.

You will write OpenMP program to determine the heat distribution in a space using synchronous iteration on a multicore. We have 2-dimensional square space and simple boundary conditions (edge points have fixed temperatures). The objective is to find the temperature distribution within. The temperature of the interior depends upon the temperature around it. We find the temperature distribution by dividing the area into a fine mesh of points, $h_{i,j}$. The temperature at an inside point is calculated as the average of the temperatures of the four neighboring points, as shown in the following figure.



The edge points are when i = 0, i = n-1, j = 0, or j = n-1, and have fixed values corresponding to the fixed temperatures of the edges. The temperature at each interior point is calculated as:

$$h_{i,j} = \frac{h_{i-1,j} + h_{i+1,j} + h_{i,j-1} + h_{i,j+1}}{4}$$

(**0 < i < n** ; **0 < j < n**, remember that edge points have fixed values)

Stopping condition: You stop after a fixed number of iterations.

Assume we have (n x n) points (including edge points), the initial situation is as follows:

- The edge points: (0,0) → (0,n-1), (0,0)→(n-1,0), (0,n1-)→(n-1, n-1), and (n-1,0) → (n-1,n-1) are initialized to 100. Note that (x,y) means row x and column y.
- All other points are initialized to zero.

Remember that the boundary points do not change across iterations.

There will be a fixed number of iterations ITR. In each iteration, each point in the grid will be updated. But be careful: The points at iteration i must use the neighbors' values of iteration i, not the updated version. That is, if you are at iteration 10, for example, and you update point (5,7). This new value of point (5,7) cannot be used to calculate the temperature of its neighbors in this iteration 10. When updating the temperature of these neighbors, use the old value of (5,7) because its new value will be used in the following iteration. So, you may need to have temporary storage for values of iteration i and you update the values in a different storage. Then, in the next iteration, the second storage becomes the new current and you update the other one.

As a help, we are providing you with a source code, heatdist.c, that contains the sequential code and the time measurement. You just need to plugin your code in that file as indicated in the comments in the file.

What you must do:

1. Read the file heatdist.c very carefully to understand how it works. Be careful that the 2D arrays are accessed as 1D array using the macro index(i,j,N). So, if you want to access array playground of dimension 100x100 (all arrays are assumed to be square) then playground[2,3] is accessed using playground[index(2, 3, 100)].
2. Do not change anything except the function parallel_heat_dist(). You can add any other function you see needed to be called by parallel_heat_dist(). The code will take care of measuring the timing.
3. SSH to one of the crunchy machines. But do all your experiments in the same crunch machine for consistency.
4. Compile the file, without any modifications from your part, first and make a run. The compilation command is: gcc -Wall -std=c99 -fopenmp -o heatdist heatdist.c -lm
5. You run the code with ./heatdist x y z k where x is the dimension of the square 2D array, y is the number of iterations and z can be 0 (runs and measures the time of the sequential version) or 1 (run the OpenMP version when you finish it and measure the timing), and k is the number of threads for the OpenMP version. Note that even if you try with z = 0 (i.e. sequential version) you must type a number for k. Otherwise, the program will exit.
6. **Be careful:** when you run your OpenMP version (i.e. z= 1) the code will take long time because it will run both the sequential and parallel to compare the results for correctness. It will measure the time of the parallel version only.

The report

Once you are done implementing your OpenMP version in file heatdist.c, do the following experiments.

At top of the report, write the following:

- Which crunchy machine did you use?
- gcc version: You can get it by typing: *gcc -v*

- Multicore characteristic. You can get that by typing: *lscpu* or check /proc/cpuinfo file. Do not copy all the information that you get from the command or file. But only: number of cores and hyperthreading technology if any.

Then do the following experiments:

Experiment 1

Fix the number of iterations to 200. For the 2D square array, test with sizes: 10, 100, 1000, and 10000. For each one of these sizes, get the time for the sequential as well as the parallel with 1, 2, 4, and 8 threads. Calculate the speedup, for each size and for each number of threads relative to the sequential version. Note that the one thread version is a bit different than the purely sequential version. So, you need to have a table like the following where the rows represent the speedup.

	10x10	100x100	1000x1000	20000 x 20000
1 thread				
2				
4				
8				

Then answer the following questions. You can use the info you got about the multicore architecture in your analysis of the results.

- What is the *pattern* that you can see from the results in the table? You need to see what happens with
  - Fixed number of threads and different problem sizes
  - Fixed problem size and different number of threads
  - What is the best (i.e. highest speedup) number of threads for each problem size.
- Explain the reason for each pattern you discovered.

Experiment 2

Fix the array dimension to 1000. The number of iterations varies as: 10, 20, 30, and 40.
For each one calculate the speedup with 1, 2, 4, and 8 threads relative to sequential and fill up a table like the following where the rows represent the speedup.

	10 iterations	20	30	40
1 thread				
2				
4				
8				

Then answer the following questions:

- What is the pattern that you can see from the results in the table? You need to see what happens with:
  - Fixed number of threads and different iterations
  - Fixed number of iterations and different number of threads
  - What is the best number of threads (in terms of speedup) for each iteration.
- Explain the reason for each pattern you discovered, as you did in experiment 1.

Note about both experiments:

- A pattern can be anything: speedup increases/decreases/stays fixed, or even sometimes increases and sometimes decreases. The important thing is that you need to have an interpretation for that.
- If, for some sizes, you run out of memory, or the experiment takes more than half an hour, please state so on the table.

What to submit:

- The source code heatdist.c [**Careful**: Do NOT submit the original file. But the file you have updated.]
- A report that contains the tables and answers to the questions mentioned above. That report's name is netID.pdf where netID is your own netID.

Put all the above in a zip file named by your netID.zip and upload them to Brightspace.